

第 3 讲

CUDA Parallel Execution Model: Multidimensional Grids and Data

目录

1 核心内容	1
1.1 并行线程数组与单程序多数据模型	1
1.2 块内协作与块间独立性	2
1.3 多维组织的目的	2
2 多维执行层次与索引系统	3
2.1 四组只读内建变量	3
2.2 一维全局索引	4
2.3 二维全局坐标	4
2.4 三维全局坐标	5
2.5 块数、每块线程数与总线程数	6
2.6 每次核函数启动拥有独立配置	7
3 多维启动配置与数据布局	7
3.1 使用 <code>dim3</code> 指定块与网格尺寸	7
3.2 向上取整除法	8
3.3 覆盖矩形与多余线程	8
3.4 二维数据的行主序布局	9
3.5 三维数据的线性化补充	11
4 实例一：RGB 彩色图像转灰度	11
4.1 数据并行分解	12
4.2 完整核函数	12
4.3 启动配置	13
4.4 一次线程索引的完整追踪	14
5 实例二：邻域平均模糊	15
5.1 从逐元素映射到邻域映射	15
5.2 核函数中的双重循环	16
5.3 两层边界条件	16
5.4 固定分母为何会产生错误	17
5.5 完整核函数	18
5.6 控制流与工作量	19
6 实例三：矩阵与矩阵乘法	19
6.1 输出元素的定义	19
6.2 简化方阵核函数	20
6.3 一般矩形矩阵	21
6.4 网格尺寸由输出矩阵决定	22
6.5 单线程工作量与访问方式	23
7 例题与完整推导	24
7.1 统一计算框架	24
7.2 例题一：一维网格与二维线程块	24
7.3 例题二：二维网格与二维线程块	25

7.4	例题三：三维网格与三维线程块	26
7.5	综合例题：最后一个有效像素与第一个无效线程	26
7.6	常见计算错误	27
8	统一方法与关键结论	27
8.1	从输出空间建立并行映射	27
8.2	三个实例的结构对照	28
8.3	关键公式汇总	29
8.4	正确性检查表	29

1 核心内容

内容概览

- 理解 CUDA 线程的多维逻辑组织；
- 了解如何在核函数中使用循环等控制结构。

1.1 并行线程数组与单程序多数据模型

CUDA 核函数启动后，由一个线程网格执行。网格中的所有线程运行同一份核函数代码，但每个线程都能读取一组与自身位置有关的只读内建变量，从而确定自己负责的数据位置。该执行方式属于单程序多数据 (single-program, multiple-data, SPMD) 模型：

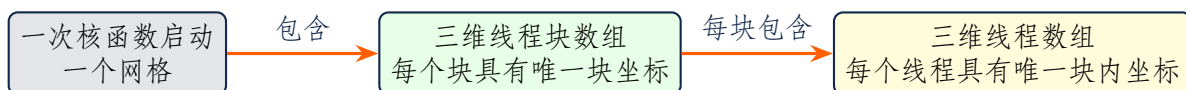
同一程序 + 不同线程标识 $\implies$ 不同数据上的并行实例。

定义

网格是一次核函数启动所创建的全部线程块的逻辑集合；线程块是能够使用块内共享机制进行协作的一组线程。因而，一次常规核函数启动形成两级层次：

网格 $\longrightarrow$ 线程块 $\longrightarrow$ 线程。

每个网格内的线程块具有相同的块尺寸，每个线程块内的线程执行相同的核函数代码。



未使用的维度尺寸取 1；所有线程执行同一核函数，并通过坐标选择不同数据。

图 1 CUDA 的逻辑执行层次

1.2 块内协作与块间独立性

线程块既是逻辑索引单位，也是协作边界。块内线程可通过共享内存 (shared memory)、原子操作 (atomic operation) 和屏障同步 (barrier synchronization) 等机制协作。不同线程块不共享块内资源，通常主要通过全局内存 (global memory) 交换数据，因此块间协作更受限制。

这一约束直接解释了线程块设计的两个基本原则：

- 同一块内的线程应对应一组存在局部数据关系、未来可能需要协作的工作项；
- 各线程块应尽量能够独立完成工作，使硬件可以按任意次序调度线程块。

1.3 多维组织的目的

CUDA 将网格和线程块统一表示为至多三维的逻辑数组。即使应用只使用一维或二维，未使用的维度仍存在，其尺寸取为 1。多维组织的主要价值是让线程坐标直接对应数据坐标：

- 一维向量通常使用全局下标 i ；
- 二维图像或矩阵通常使用行列坐标 (r, c) ；
- 三维体数据通常使用坐标 (z, y, x) 或 (d, r, c) 。

这种对应关系降低了地址计算的复杂度，也使边界判断与任务划分更清晰。多维网格并未改变存储器本身的一维线性地址空间；它提供的是更自然的逻辑坐标系。逻辑坐标仍需在线程内部转换为线性内存偏移。

关键结论

多维线程组织解决的是“线程负责哪个逻辑数据元素”；行主序等数据布局解决的是“该逻辑元素位于线性内存的哪个位置”。二者需要连续完成：先求全局逻辑坐标，再求线性地址。

2 多维执行层次与索引系统

2.1 四组只读内建变量

每个正在执行的线程都可读取四组 CUDA 只读内建变量。它们共同给出“网格有多大、当前块在哪里、块有多大、当前线程在块内哪里”。

表 1 多维线程组织中的四组内建变量

变量	含义	三个分量及取值范围
<code>gridDim</code>	网格在各方向上的线程块数量	<code>x</code> 、 <code>y</code> 、 <code>z</code> ；每个分量都是相应方向上的块数
<code>blockIdx</code>	当前线程块在网格中的坐标	$0 \leq \text{blockIdx}.d < \text{gridDim}.d$ ，其中 $d \in \{x, y, z\}$
<code>blockDim</code>	每个线程块在各方向上的线程数量	<code>x</code> 、 <code>y</code> 、 <code>z</code> ；三者乘积为每块线程总数
<code>threadIdx</code>	当前线程在线程块内部的坐标	$0 \leq \text{threadIdx}.d < \text{blockDim}.d$ ，其中 $d \in \{x, y, z\}$

其中，`gridDim` 与 `blockDim` 描述尺寸，`blockIdx` 与 `threadIdx` 描述位置。对一维配置，未使用的 `y, z` 维尺寸为 1，对应索引只能取 0；对二维配置，未使用的 `z` 维尺寸为 1。

三维尺寸示例

若

$$\text{gridDim} = (8, 3, 2), \quad \text{blockDim} = (5, 4, 3),$$

则网格含有

$$8 \times 3 \times 2 = 48$$

个线程块，每块含有

$$5 \times 4 \times 3 = 60$$

个线程，总启动线程数为

$$48 \times 60 = 2880.$$

块坐标的最大值为 (7, 2, 1)，块内线程坐标的最大值为 (4, 3, 2)。

2.2 一维全局索引

仅使用线程的块内下标 `threadIdx.x` 无法唯一标识整个网格中的线程，因为不同块内可以存在相同的块内下标。设每块包含 $B_x = \text{blockDim.x}$ 个线程，当前块编号为 $b_x = \text{blockIdx.x}$ ，当前线程在块内的编号为 $t_x = \text{threadIdx.x}$ 。当前块之前共有 b_x 个完整线程块，因此它们共覆盖 $b_x B_x$ 个工作项；再加上块内偏移 t_x ，得到一维全局工作下标

$$i = b_x B_x + t_x.$$

对应的 CUDA 表达式为

代码 1 一维全局工作下标

```
1 unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
```

若网格在 x 方向有 $G_x = \text{gridDim.x}$ 个块，则启动线程覆盖的全局下标范围为

$$0 \leq i < G_x B_x.$$

数据长度通常并不恰好等于 $G_x B_x$ ，因此实际内存访问仍需检查 i 是否小于数据长度。

2.3 二维全局坐标

二维数据通常用行列坐标 (r, c) 表示。CUDA 习惯让 x 方向对应列，让 y 方向对应行。设

$$\begin{aligned} b_x &= \text{blockIdx.x}, & B_x &= \text{blockDim.x}, & t_x &= \text{threadIdx.x}, \\ b_y &= \text{blockIdx.y}, & B_y &= \text{blockDim.y}, & t_y &= \text{threadIdx.y}. \end{aligned}$$

则线程负责的全局行列坐标为

$$r = b_y B_y + t_y, \quad c = b_x B_x + t_x$$

即

代码 2 二维线程坐标到全局数据坐标的映射

```
1 unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
2 unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;
```

这两个式子彼此独立： x 方向只决定列坐标， y 方向只决定行坐标。其几何意义与一维情形相同，都是“前面完整块的跨度加上块内偏移”。

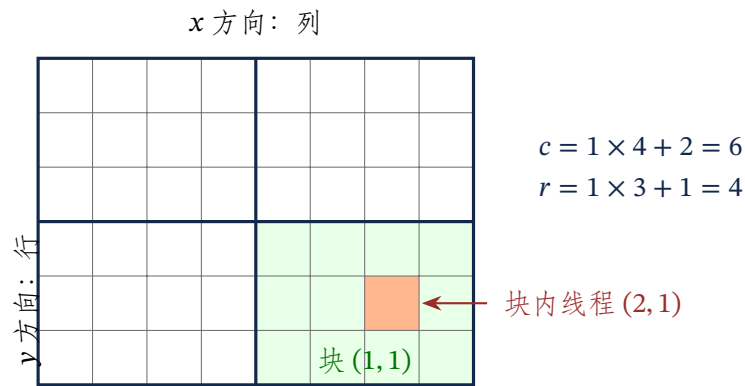


图 2 二维索引示例: 块尺寸为 4×3 , 高亮线程的全局坐标为 $(r, c) = (4, 6)$

关键结论

在二维图像或矩阵核函数中, 最常见的坐标约定是

$$x \leftrightarrow \text{列}, \quad y \leftrightarrow \text{行}.$$

因此, 宽度与 x 方向相关, 高度与 y 方向相关。将二者交换会导致转置式访问、错误边界判断或越界。

2.4 三维全局坐标

对体数据或三维张量, 可逐维应用同一构造。若把 z 方向解释为深度, 则

$$\begin{aligned} c &= \text{blockIdx.x} \text{blockDim.x} + \text{threadIdx.x}, \\ r &= \text{blockIdx.y} \text{blockDim.y} + \text{threadIdx.y}, \\ d &= \text{blockIdx.z} \text{blockDim.z} + \text{threadIdx.z}. \end{aligned}$$

每个线程因此对应一个逻辑体素 (d, r, c) 。各维度的计算互不干扰, 边界条件也应逐维检查。

2.5 块数、每块线程数与总线程数

记网格尺寸为

$$(G_x, G_y, G_z) = (\text{gridDim.x}, \text{gridDim.y}, \text{gridDim.z}),$$

块尺寸为

$$(B_x, B_y, B_z) = (\text{blockDim.x}, \text{blockDim.y}, \text{blockDim.z}).$$

则

$$N_{\text{blocks}} = G_x G_y G_z,$$

$$N_{\text{threads/block}} = B_x B_y B_z,$$

$$N_{\text{threads,total}} = G_x G_y G_z B_x B_y B_z.$$

这些乘积计算的是启动的线程数量。有效工作项数量由数据尺寸决定，边界处多出的线程会通过条件判断退出。

三维逻辑坐标的线性化

多维线程索引可逐维使用；在需要给全部启动线程构造一个线性序号时，可进一步线性化。令

$$N_x = G_x B_x, \quad N_y = G_y B_y,$$

并用 (g_z, g_y, g_x) 表示线程的三维全局坐标，则按 x 最快变化的顺序，线性线程坐标可写为

$$q = (g_z N_y + g_y) N_x + g_x.$$

该式只是坐标表示的补充。处理二维数据时，直接保留 (r, c) 通常更清晰，无需先构造 q 再反解行列。

2.6 每次核函数启动拥有独立配置

同一应用中的多个核函数可以使用彼此独立的网格尺寸与块尺寸。配置属于某一次核函数启动，不是整个程序的固定属性。图像处理阶段可以采用二维组织，向量阶段可以采用一维组织，体数据计算可以采用三维组织。只要每次启动时传入的配置与该核函数的工作项形状一致即可。

3 多维启动配置与数据布局

3.1 使用 **dim3** 指定块与网格尺寸

核函数启动语法中的两个配置参数分别表示网格尺寸和线程块尺寸：

代码 3 二维网格的典型启动配置

```
1 dim3 numThreadsPerBlock(32, 32);  
2  
3 dim3 numBlocks(  
4     (width + numThreadsPerBlock.x - 1) / numThreadsPerBlock.x,  
5     (height + numThreadsPerBlock.y - 1) / numThreadsPerBlock.y);  
6  
7 kernel<<<numBlocks, numThreadsPerBlock>>>(/* kernel arguments */);
```

内建类型 **dim3** 用三个分量保存 x, y, z 方向的尺寸。上例只显式给出两个分量，因此 z 分量取 1。核函数内部读到的 **blockDim** 等于启动时传入的 **numThreadsPerBlock**，读到的 **gridDim** 等于 **numBlocks**。

二维数据的启动配置遵循以下对应关系：

$$\begin{aligned} B_x &= \text{线程块宽度}, & B_y &= \text{线程块高度}, \\ G_x &= \text{水平方向块数}, & G_y &= \text{竖直方向块数}. \end{aligned}$$

其中 $B_x B_y$ 是每块线程数， $G_x G_y$ 是网格中的块数。

3.2 向上取整除法

若数据宽度为 W ，每块在 x 方向覆盖 B_x 个元素，则所需块数必须满足

$$G_x B_x \geq W.$$

取满足该条件的最小整数，得到

$$G_x = \left\lceil \frac{W}{B_x} \right\rceil, \quad G_y = \left\lceil \frac{H}{B_y} \right\rceil.$$

对正整数，可用整数运算实现为

$$\left\lceil \frac{W}{B_x} \right\rceil = \frac{W + B_x - 1}{B_x},$$

右侧除法按整数除法截断。

整数向上取整公式为何成立

令 $W = qB_x + r$ ，其中 $0 \leq r < B_x$ 。

- 若 $r = 0$ ，则 $(W + B_x - 1)/B_x = q + (B_x - 1)/B_x$ ，整数除法结果为 q ；
- 若 $r > 0$ ，则 $W + B_x - 1 = (q + 1)B_x + (r - 1)$ ，整数除法结果为 $q + 1$ 。

两种情形分别对应整除与非整除，恰好等于 $\lceil W/B_x \rceil$ 。

3.3 覆盖矩形与多余线程

向上取整保证所有有效元素都被覆盖，但通常会启动一部分落在数据范围外的线程。以宽 $W = 76$ 、高 $H = 62$ 的图像和 16×16 线程块为例：

$$G_x = \left\lceil \frac{76}{16} \right\rceil = 5, \quad G_y = \left\lceil \frac{62}{16} \right\rceil = 4.$$

因此启动 $5 \times 4 = 20$ 个线程块，共有

$$20 \times 16 \times 16 = 5120$$

个线程，而图像只有

$$76 \times 62 = 4712$$

个像素。多出的 $5120 - 4712 = 408$ 个线程必须通过边界条件排除。

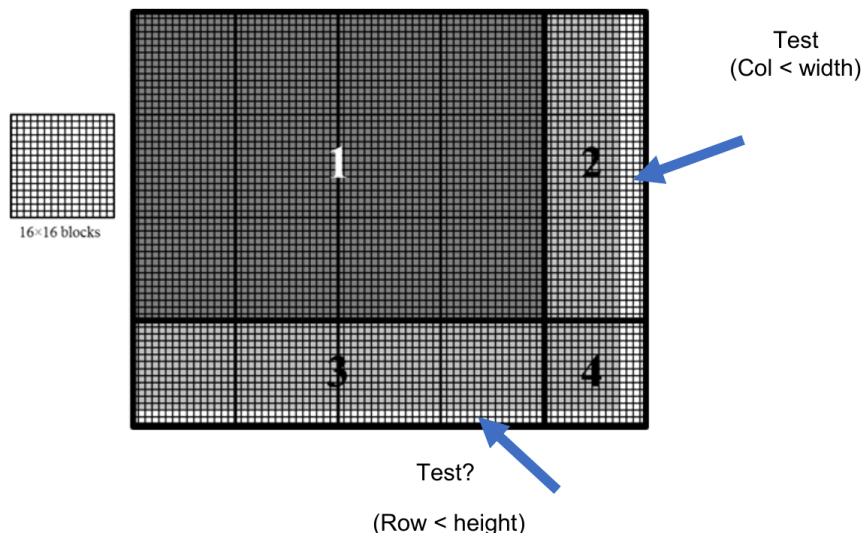


图 3 76×62 图像由 16×16 线程块向上取整覆盖。右侧、底部以及右下角区域包含越过图像边界的线程。

对应的二维有效性条件为

代码 4 二维输出坐标的边界保护

```
1 if (row < height && col < width) {
2     // All output memory accesses using (row, col) are valid here.
3 }
```

需要特别注意

向上取整只保证“线程数量足够”，不保证每个线程都对应有效数据。只要数据尺寸可能不是块尺寸的整数倍，就必须在访问内存前检查每个相关维度。

3.4 二维数据的行主序布局

C 与 C++ 按行主序 (row-major order) 存放普通二维数组：同一行的元素在线性内存中连续，整行存完后才存下一行。设矩阵或图像共有 H 行、 W 列，元素的逻辑坐标为 (r, c) ，其中

$$0 \leq r < H, \quad 0 \leq c < W.$$

第 r 行之前共有 r 个完整行，每行含 W 个元素，因此该行起点的线性偏移为 rW ；再加上行内偏移 c ，得到

$$\text{index}(r, c) = rW + c.$$

若数组首地址为 p 、元素类型为 T ，对应字节地址为

$$\text{addr}(r, c) = p + (rW + c) \text{sizeof}(T).$$

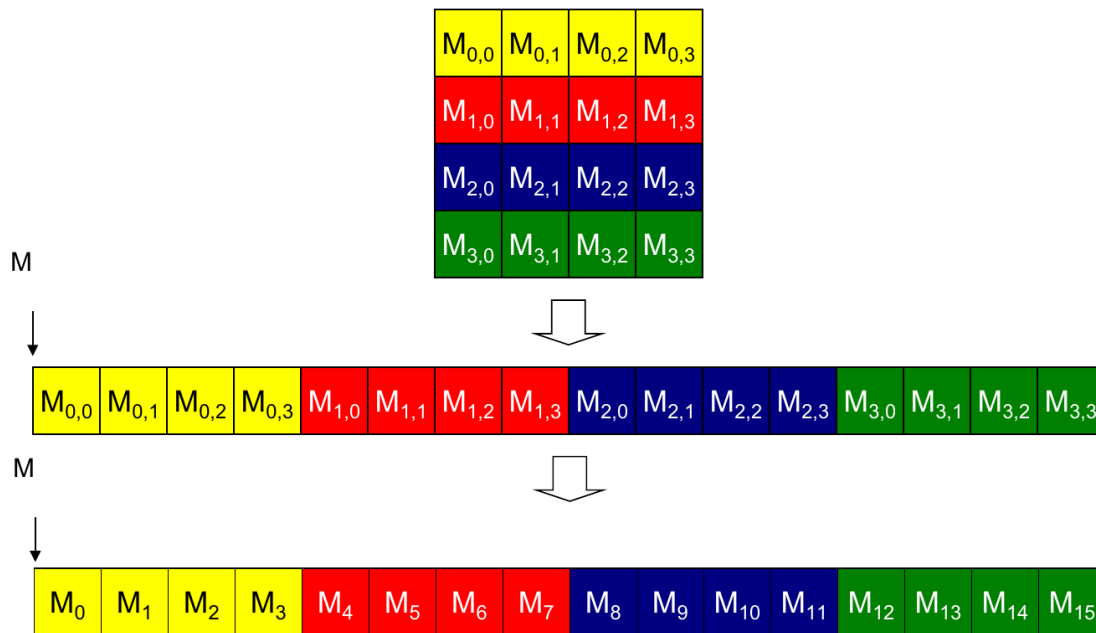


图 4 4×4 矩阵按行主序展开为一维连续序列。同行元素相邻，下一行紧随上一行。

从二维坐标到线性下标

对宽度 $W = 4$ 的矩阵，元素 $M_{2,1}$ 位于第 2 行、第 1 列，采用从 0 开始的下标。其线性下标为

$$\text{index}(2, 1) = 2 \times 4 + 1 = 9.$$

这意味着它是从线性下标 0 开始计数的第 10 个元素。

行主序公式还直接给出两个局部关系：

$$\text{index}(r, c + 1) = \text{index}(r, c) + 1,$$

说明同一行相邻列在内存中相邻；而

$$\text{index}(r + 1, c) = \text{index}(r, c) + W,$$

说明同一列相邻行之间相隔 W 个元素。

3.5 三维数据的线性化补充

若三维数据尺寸为深度 D 、高度 H 、宽度 W ，并采用 x 对应列、 y 对应行、 z 对应深度的约定，则可先将 (d, r, c) 看成第 d 个二维平面中的位置。每个平面含 HW 个元素，因此

$$\text{index}(d, r, c) = (dH + r)W + c = dHW + rW + c.$$

该式是二维行主序的逐层扩展。核函数仍应先验证

$$0 \leq d < D, \quad 0 \leq r < H, \quad 0 \leq c < W,$$

再使用线性下标访问内存。

关键结论

二维核函数的标准地址链条为

$$(\text{blockIdx}, \text{threadIdx}) \longrightarrow (r, c) \longrightarrow rW + c \longrightarrow \text{内存访问}.$$

每一步使用不同信息：线程层次给出逻辑坐标，数据布局给出线性偏移，边界条件保证该偏移有效。

4 实例一：RGB 彩色图像转灰度

4.1 数据并行分解

红绿蓝 (red-green-blue, RGB) 图像的每个像素包含三个颜色分量。三个独立的一维数组 **red**、**green** 和 **blue** 分别保存三个输入分量，数组 **gray** 保存灰度输出。四个数组的长度均为 WH ，其中 W 为图像宽度， H 为图像高度。

每个输出像素只依赖同一位置的三个颜色分量，不依赖其他像素。因此可把输出空间直接作为并行工作空间：

$$\boxed{\text{一个线程} \longleftrightarrow \text{一个输出像素}}.$$

线程在二维网格中先求自己负责的 (r, c) ，再用 $p = rW + c$ 访问三个输入通道并写入一个输出值。

设某像素的红、绿、蓝分量分别为 R_p, G_p, B_p ，灰度计算为

$$\boxed{Y_p = \frac{3}{10}R_p + \frac{6}{10}G_p + \frac{1}{10}B_p}.$$

三个系数之和为 1，因此当三个输入均位于 $[0, 255]$ 时，浮点中间结果也位于该区间内。

单个像素的灰度计算

若某像素满足

$$R_p = 100, \quad G_p = 150, \quad B_p = 200,$$

则

$$Y_p = 0.3 \times 100 + 0.6 \times 150 + 0.1 \times 200 = 30 + 90 + 20 = 140.$$

该像素的三个输入值由一个线程读取，结果 140 由同一线程写入对应的灰度位置。

4.2 完整核函数

核函数使用三个独立通道数组与上述权重。线性下标 **pix** 在边界判断通过后计算，使边界保护与内存访问的对应关系更直接。

代码 5 带二维边界保护的 RGB 转灰度核函数

```

1 __global__
2 void rgb2gray_kernel(unsigned char* red,
3                     unsigned char* green,
4                     unsigned char* blue,
5                     unsigned char* gray,
6                     unsigned int width,
7                     unsigned int height)
8 {
9     unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
10    unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;
11
12    if (row < height && col < width) {
13        unsigned int pix = row * width + col;
14
15        gray[pix] = red[pix]    * 3.0f / 10.0f
16                  + green[pix] * 6.0f / 10.0f
17                  + blue[pix]  * 1.0f / 10.0f;
18    }
19 }

```

后缀 **f** 表示单精度浮点字面量。乘加表达式先产生浮点中间结果，再在赋给 **unsigned char** 时转换为字节值。源实现未加入额外舍入步骤，因此若结果带小数部分，赋值转换会去除其小数部分。

4.3 启动配置

采用 16×16 线程块时，二维启动配置可写为

代码 6 RGB 转灰度核函数的二维启动配置

```

1 dim3 threadsPerBlock(16, 16);
2 dim3 numBlocks(
3     (width + threadsPerBlock.x - 1) / threadsPerBlock.x,
4     (height + threadsPerBlock.y - 1) / threadsPerBlock.y);
5
6 rgb2gray_kernel<<<numBlocks, threadsPerBlock>>>(
7     red, green, blue, gray, width, height);

```

该配置创建的矩形线程区域至少覆盖整个图像。右边界与下边界多出的线程仍会执行到条件判断，但不会读取或写入任何像素。

4.4 一次线程索引的完整追踪

考虑宽 $W = 76$ 、高 $H = 62$ 的图像，块尺寸为 16×16 。某线程满足

$$\text{blockIdx} = (3, 2), \quad \text{threadIdx} = (5, 7).$$

其全局列、行坐标分别为

$$c = 3 \times 16 + 5 = 53,$$

$$r = 2 \times 16 + 7 = 39.$$

因为 $53 < 76$ 且 $39 < 62$ ，该线程对应有效像素。线性下标为

$$p = 39 \times 76 + 53 = 3017.$$

于是该线程读取

$$\text{red}[3017], \quad \text{green}[3017], \quad \text{blue}[3017],$$

并写入 $\text{gray}[3017]$ 。

若同一块中的线程改为 $\text{threadIdx.y} = 15$ ，则行坐标变为

$$r = 2 \times 16 + 15 = 47,$$

仍然有效；而位于最后一行块中的部分线程会得到 $r \geq 62$ ，这些线程由外层条件排除。

关键结论

该实例包含一条完整的二维逐元素模式：

二维线程坐标

↓

像素行列坐标 (r, c)

↓

线性下标 $p = rW + c$

↓

三个输入读取与一个输出写入。

所有四次内存访问共享同一个有效性条件 $r < H \wedge c < W$ 。

5 实例二：邻域平均模糊

5.1 从逐元素映射到邻域映射

灰度图像模糊仍采用“一个线程生成一个输出像素”的并行分解，但一个输出不再只依赖一个输入位置。设模糊半径为

$$R = \text{BLUR_SIZE}.$$

输出像素 (r, c) 的名义邻域为

$$\mathcal{N}_R(r, c) = \{(i, j) \mid r - R \leq i \leq r + R, c - R \leq j \leq c + R\}.$$

若该邻域完全位于图像内部，其中包含

$$(2R + 1)^2$$

个输入像素。当 $R = 1$ 时，内部像素使用 3×3 邻域，共 9 个输入。

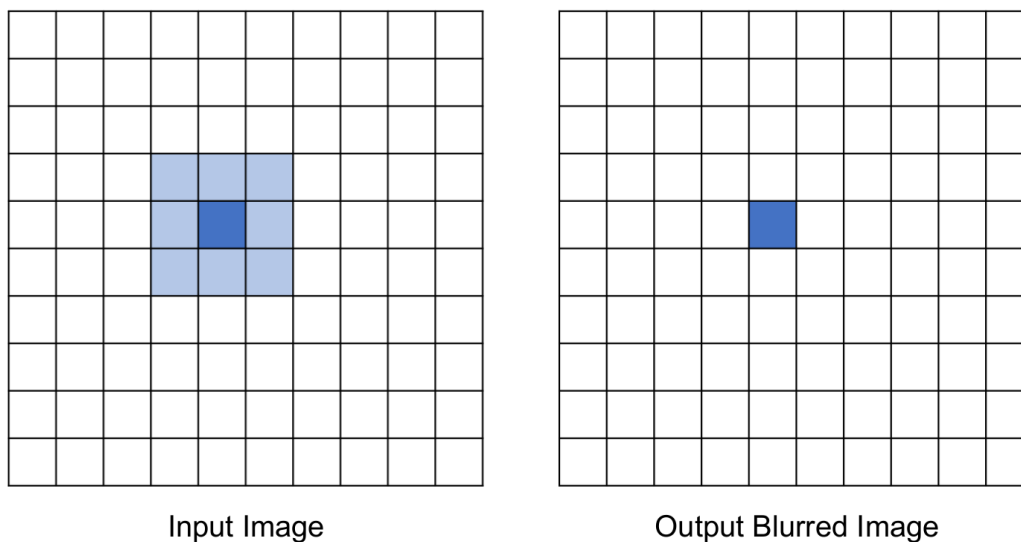


图 5 一个线程负责一个输出像素，并读取该位置周围的多个输入像素。图中半径为 1。

设图像有效区域为

$$\Omega = \{(i, j) \mid 0 \leq i < H, 0 \leq j < W\}.$$

真正参与边界像素计算的集合是

$$\mathcal{V}_R(r, c) = \mathcal{N}_R(r, c) \cap \Omega.$$

因此输出应定义为有效邻域的平均值：

$$B_{r,c} = \frac{1}{|\mathcal{V}_R(r, c)|} \sum_{(i,j) \in \mathcal{V}_R(r, c)} I_{i,j}.$$

5.2 核函数中的双重循环

每个线程知道自己的输出坐标 (`outRow, outCol`) 后，需要枚举邻域的所有候选输入坐标。二维邻域自然对应两层循环：外层改变输入行，内层改变输入列。

代码 7 未处理输入边界的基本邻域遍历

```

1 unsigned int sum = 0;
2
3 for (int inRow = outRow - BLUR_SIZE;
4     inRow < outRow + BLUR_SIZE + 1;
5     ++inRow) {
6     for (int inCol = outCol - BLUR_SIZE;
7         inCol < outCol + BLUR_SIZE + 1;
8         ++inCol) {
9         sum += image[inRow * width + inCol];
10    }
11 }
```

该代码只对完全位于图像内部的输出像素安全。即使输出坐标本身有效，邻域中的 `inRow` 或 `inCol` 仍可能越界。例如，左上角输出 (0,0) 在 $R = 1$ 时会尝试访问行 -1 或列 -1 。

5.3 两层边界条件

模糊核函数中存在两类不同的内存访问，因而需要两层保护：

1. **输出保护**：验证线程负责的输出坐标满足 $0 \leq \text{outRow} < H$ 且 $0 \leq \text{outCol} < W$ ，从而保证最终写入有效；
2. **输入保护**：对循环中每一个候选邻域坐标验证 $0 \leq \text{inRow} < H$ 且 $0 \leq \text{inCol} < W$ ，从而保证每次读取有效。

内存访问规则可形式化为：

每次内存访问都必须有对应的索引保护

对任意数组访问 `array[index]`，必须能够指出一条在该访问之前成立的条件，并由该条件推出 `index` 位于数组合法范围内。模糊核函数的输出写入由外层条件保护，输入读取由内层条件保护。

邻域索引使用有符号整数 `int`。这样，顶边和左边附近的候选坐标可以自然取负值，并由 `inRow >= 0` 与 `inCol >= 0` 排除。若这些变量使用无符号类型，减去模糊半径可能发生下溢，使边界逻辑难以按预期工作。

5.4 固定分母为何会产生错误

只跳过越界输入还不够。若始终除以名义邻域大小 $(2R+1)^2$ ，边界处没有被加入求和的像素仍被隐含地计入分母，相当于把缺失像素按 0 处理，输出会偏小。

当 $R = 1$ 时：

- 内部像素有 9 个有效邻居，分母为 9；
- 非角点边缘像素通常有 6 个有效邻居，分母应为 6；
- 角点像素有 4 个有效邻居，分母应为 4。

因此必须在累加有效像素时同步维护计数 `count`，最后使用 `sum / count`。

角点像素的有效邻域

考虑 3×3 灰度块

$$\begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{bmatrix}.$$

当 $R = 1$ 时，中心像素的九点平均为 50。左上角输出只使用

$$10, 20, 40, 50,$$

所以

$$\text{sum} = 120, \quad \text{count} = 4, \quad B_{0,0} = 120/4 = 30.$$

若错误地除以 9，则得到 13，明显把边界亮度压低。

5.5 完整核函数

代码 8 对有效邻域求平均的完整灰度模糊核函数

```
1 #define BLUR_SIZE 1
2
3 __global__
4 void blur_kernel(unsigned char* image,
5                 unsigned char* blurred,
6                 unsigned int width,
7                 unsigned int height)
8 {
9     int outRow = blockIdx.y * blockDim.y + threadIdx.y;
10    int outCol = blockIdx.x * blockDim.x + threadIdx.x;
11
12    if (outRow < height && outCol < width) {
13        unsigned int sum = 0;
14        unsigned int count = 0;
15
16        for (int inRow = outRow - BLUR_SIZE;
17             inRow < outRow + BLUR_SIZE + 1;
18             ++inRow) {
19            for (int inCol = outCol - BLUR_SIZE;
20                 inCol < outCol + BLUR_SIZE + 1;
21                 ++inCol) {
22                if (inRow >= 0 && inRow < height &&
23                    inCol >= 0 && inCol < width) {
24                    sum += image[inRow * width + inCol];
25                    ++count;
26                }
27            }
28        }
29
30        blurred[outRow * width + outCol]
31            = (unsigned char)(sum / count);
32    }
33 }
```

对任何通过外层条件的线程，中心输入(outRow,outCol)一定有效，因此 count 至少为 1，最终除法不会除以 0。

5.6 控制流与工作量

所有线程执行同一双重循环结构，但边界线程的内层条件只有部分迭代成立。对半径 R ，每个有效输出线程检查 $(2R + 1)^2$ 个候选位置；实际读取次数等于 $|\mathcal{V}_R(r, c)|$ 。这说明核函数内部可以包含普通控制结构：循环负责枚举一个线程的局部工作集，条件判断负责过滤无效位置。

关键结论

RGB 转灰度与模糊使用相同的输出并行分解，但单线程输入工作集不同：

计算	每线程输出	每线程输入读取
RGB 转灰度	1 个灰度像素	3 个同位置通道值
邻域模糊	1 个模糊像素	$ \mathcal{V}_R(r, c) $ 个邻域像素

线程数量由输出空间决定，单线程内部的循环由输入依赖范围决定。

6 实例三：矩阵与矩阵乘法

6.1 输出元素的定义

设方阵 $A, B \in \mathbb{R}^{N \times N}$ ，其乘积记为

$$C = AB.$$

采用从 0 开始的下标，输出元素 $C_{r,c}$ 是 A 的第 r 行与 B 的第 c 列的内积：

$$C_{r,c} = \sum_{k=0}^{N-1} A_{r,k} B_{k,c}.$$

固定 (r, c) 后，求和变量 k 枚举共享维度。不同输出位置写入 C 的不同元素，因此可以并行计算：

$$\boxed{\text{一个线程} \longleftrightarrow \text{一个输出矩阵元素}}.$$

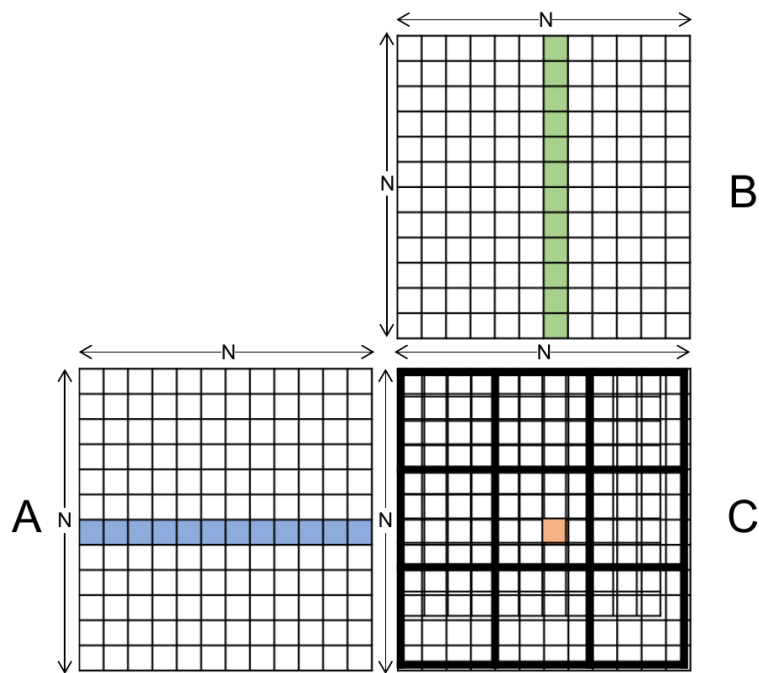


图 6 输出元素由 A 的一行与 B 的一列做逐项乘积并求和得到；二维线程网格覆盖输出矩阵。

这种分解与图像模糊具有相同的外层结构：线程坐标决定一个输出位置，线程内部再用循环遍历生成该输出所需的多个输入。区别在于，模糊遍历二维局部邻域，矩阵乘法遍历长度为 N 的共享维度。

6.2 简化方阵核函数

方阵情形可使用以下简化实现：

代码 9 简化的方阵乘法核函数

```

1 __global__
2 void mm_kernel(float* A, float* B, float* C,
3               unsigned int N)
4 {
5     unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
6     unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;
7
8     float sum = 0.0f;
9     for (unsigned int k = 0; k < N; ++k) {
10         sum += A[row * N + k] * B[k * N + col];
11     }
12
13     C[row * N + col] = sum;
14 }

```

代码中三个线性下标分别对应

$$\begin{aligned}
 A_{r,k} &\longleftrightarrow A[\text{row} * N + k], \\
 B_{k,c} &\longleftrightarrow B[k * N + \text{col}], \\
 C_{r,c} &\longleftrightarrow C[\text{row} * N + \text{col}].
 \end{aligned}$$

该简化版本未包含边界条件。只有在启动线程区域恰好不超过 $N \times N$ 输出范围，或调用者另有保证时，所有访问才有效；一般实现必须显式处理边界。

6.3 一般矩形矩阵

矩阵不必为方阵。设

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}.$$

两者共享维度均为 K ，输出为

$$C = AB \in \mathbb{R}^{M \times N},$$

并满足

$$C_{r,c} = \sum_{k=0}^{K-1} A_{r,k} B_{k,c}, \quad 0 \leq r < M, \quad 0 \leq c < N.$$

这里三个矩阵的行主序宽度不同： A 的每行宽度为 K ， B 与 C 的每行宽度为 N 。因此正

确线性下标为

$$\text{index}_A(r, k) = rK + k,$$

$$\text{index}_B(k, c) = kN + c,$$

$$\text{index}_C(r, c) = rN + c.$$

把所有宽度都写成同一个 N 只适用于方阵情形。

带边界条件的矩形版本

将维度关系推广为 $M \times K$ 、 $K \times N$ 和 $M \times N$ ，并加入边界条件，可得到：

```

1  __global__
2  void mm_kernel_rect(float* A, float* B, float* C,
3                      unsigned int M,
4                      unsigned int K,
5                      unsigned int N)
6  {
7      unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
8      unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;
9
10     if (row < M && col < N) {
11         float sum = 0.0f;
12         for (unsigned int k = 0; k < K; ++k) {
13             sum += A[row * K + k] * B[k * N + col];
14         }
15         C[row * N + col] = sum;
16     }
17 }
```

外层条件同时保护 A 的行索引、 B 的列索引与 C 的写入位置；循环条件 $k < K$ 保护共享维度上的两次读取。

6.4 网格尺寸由输出矩阵决定

由于每个线程生成一个 C 元素，线程网格应覆盖 $M \times N$ 输出空间。对块尺寸 (B_x, B_y) ，启动块数为

$$G_x = \left\lceil \frac{N}{B_x} \right\rceil, \quad G_y = \left\lceil \frac{M}{B_y} \right\rceil.$$

注意 x 方向对应输出列数 N ， y 方向对应输出行数 M ；共享维度 K 决定每个线程的循环长度，不决定二维网格的外部尺寸。

代码 10 矩形矩阵乘法的启动配置

```

1 dim3 threadsPerBlock(16, 16);
2 dim3 numBlocks(
3     (N + threadsPerBlock.x - 1) / threadsPerBlock.x,
4     (M + threadsPerBlock.y - 1) / threadsPerBlock.y);
5
6 mm_kernel_rect<<<numBlocks, threadsPerBlock>>>(A, B, C, M, K, N);

```

输出空间决定启动空间

若 A 为 5×3 , B 为 3×4 , 则 C 为 5×4 。采用 2×2 线程块时,

$$G_x = \left\lceil \frac{4}{2} \right\rceil = 2, \quad G_y = \left\lceil \frac{5}{2} \right\rceil = 3.$$

共启动 2×3 个块、24 个线程, 其中 20 个线程对应有效输出元素, 最后一行块中有 4 个线程因 $r \geq 5$ 被排除。每个有效线程执行 $K = 3$ 次乘加迭代。

6.5 单线程工作量与访问方式

每个输出元素的内积包含 K 次乘法和 $K - 1$ 次数学求和, 核函数循环执行 K 次累加。全部 MN 个输出元素的算术规模随 MNK 增长。

从行主序下标可直接观察两种输入访问:

- 当 k 增加 1 时, $A[\text{row} * K + k]$ 的下标也增加 1, 线程沿 A 的一行连续读取;
- 当 k 增加 1 时, $B[k * N + \text{col}]$ 的下标增加 N , 线程沿 B 的一列跨行读取。

相邻输出线程会重复需要 A 的同一行片段或 B 的同一列片段, 但简化核函数直接从全局内存逐次读取, 没有显式利用这些重用关系。因此, 该实现的主要问题是性能: 计算单元可能花费大量时间等待数据, 形成以内存访问为主的瓶颈。

关键结论

矩阵乘法的三个维度承担不同角色:

维度	作用
M	输出行数, 也决定网格的 y 向覆盖范围
N	输出列数, 也决定网格的 x 向覆盖范围
K	内积长度, 也决定每个线程的循环次数

混淆这些维度会直接造成错误的启动配置或错误的行主序地址。

7 例题与完整推导

7.1 统一计算框架

对三维网格尺寸

$$(G_x, G_y, G_z)$$

与三维块尺寸

$$(B_x, B_y, B_z),$$

依次计算

$$N_{\text{threads/block}} = B_x B_y B_z,$$

$$N_{\text{blocks}} = G_x G_y G_z,$$

$$N_{\text{threads,total}} = N_{\text{threads/block}} N_{\text{blocks}}.$$

未使用的维度取 1，不能把它们从概念上混成 0；尺寸为 0 会意味着不启动相应工作，而一维或二维配置的剩余维度实际为 1。

7.2 例题一：一维网格与二维线程块

题目

一个 CUDA 核函数启动 32 个线程块，每个线程块包含 32×32 个线程。求每块线程数与总启动线程数。

网格可写为

$$\text{gridDim} = (32, 1, 1),$$

线程块可写为

$$\text{blockDim} = (32, 32, 1).$$

因此每块线程数为

$$N_{\text{threads/block}} = 32 \times 32 \times 1 = 1024.$$

网格中的块数为

$$N_{\text{blocks}} = 32 \times 1 \times 1 = 32.$$

总线程数为

$$N_{\text{threads,total}} = 1024 \times 32 = 32768.$$

答案

每个线程块包含 1024 个线程；整个网格共启动 32768 个线程。

7.3 例题二：二维网格与二维线程块

题目

线程块配置为 16×16 ，网格尺寸为 32×16 个块。求每块线程数、块总数与整个网格的线程总数。

对应尺寸为

$$\text{blockDim} = (16, 16, 1), \quad \text{gridDim} = (32, 16, 1).$$

每块线程数为

$$N_{\text{threads/block}} = 16 \times 16 \times 1 = 256.$$

块总数为

$$N_{\text{blocks}} = 32 \times 16 \times 1 = 512.$$

因此总线程数为

$$N_{\text{threads,total}} = 256 \times 512 = 131072.$$

也可一次性验证：

$$(32 \times 16) \times (16 \times 16) = 131072.$$

答案

每块包含 256 个线程；网格含 512 个块；整个网格共启动 131072 个线程。

7.4 例题三：三维网格与三维线程块

题目

网格尺寸为 $8 \times 4 \times 2$ 个块，每个线程块尺寸为 $4 \times 8 \times 8$ 。求每块线程数与整个网格的线程总数。

每块线程数为

$$N_{\text{threads/block}} = 4 \times 8 \times 8 = 256.$$

网格中的块数为

$$N_{\text{blocks}} = 8 \times 4 \times 2 = 64.$$

总线程数为

$$N_{\text{threads,total}} = 256 \times 64 = 16384.$$

答案

每块包含 256 个线程；网格含 64 个块；整个网格共启动 16384 个线程。

7.5 综合例题：最后一个有效像素与第一个无效线程

考虑 76×62 图像，块尺寸为 16×16 ，因此

$$\text{gridDim} = (5, 4, 1).$$

图像最后一个有效像素坐标为

$$(r, c) = (61, 75).$$

取最后一个线程块

$$\text{blockIdx} = (4, 3),$$

并取块内线程

$$\text{threadIdx} = (11, 13).$$

其坐标为

$$c = 4 \times 16 + 11 = 75,$$

$$r = 3 \times 16 + 13 = 61.$$

因此该线程恰好负责最后一个有效像素，线性下标为

$$p = 61 \times 76 + 75 = 4711,$$

与长度 $76 \times 62 = 4712$ 的数组最后下标一致。

若只把块内 x 坐标增加 1，即取 `threadIdx.x=12`，则

$$c = 4 \times 16 + 12 = 76.$$

此时 $c < 76$ 不成立，该线程是最靠近右边界的无效线程之一。这个例子说明，线程的块内索引本身完全合法，并不意味着它对应的数据坐标合法；数据边界必须单独判断。

7.6 常见计算错误

- 把“网格中的块数”与“每块线程数”混为同一个量；
- 只乘二维图中显式写出的分量，却忘记未使用维度应乘以 1；
- 把 `gridDim` 当成线程总数，或把 `blockDim` 当成块总数；
- 认为启动线程总数必然等于有效数据元素数，忽略向上取整产生的边界线程；
- 在二维索引中把 x 当作行、 y 当作列，导致宽度与高度的判断互换。

8 统一方法与关键结论

8.1 从输出空间建立并行映射

三个实例都可用同一建模步骤构造。

1. **确定输出空间。** 向量输出是一维，图像与矩阵输出通常是二维，体数据输出可以是三维。
2. **确定单线程输出。** 一个线程生成一个输出元素，使不同线程写入互不重叠的位置。
3. **选择线程块形状。** 块的逻辑维度与输出坐标轴对应，二维情形通常令 x 对应列、 y 对应行。

4. 向上取整计算网格。每一维都用数据长度除以对应块尺寸并向上取整。
5. 计算全局逻辑坐标。逐维使用“块坐标乘块尺寸加块内线程坐标”。
6. 建立边界保护。输出写入与每一类输入读取都必须由可证明其下标有效的条件保护。
7. 线性化并计算。按行主序把逻辑坐标转换为线性偏移，再执行逐元素运算或线程内部循环。

二维输出核函数的通用骨架为

代码 11 二维逐输出元素核函数骨架

```

1  __global__
2  void kernel(/* arrays and parameters */, unsigned int width,
3             unsigned int height)
4  {
5      unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
6      unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;
7
8      if (row < height && col < width) {
9          unsigned int out = row * width + col;
10
11          // Read the valid input working set for (row, col).
12          // Compute one output value.
13          // Write the result to output[out].
14      }
15  }

```

该骨架只规定输出映射。输入工作集由具体算法决定：可以是同位置的若干通道、一个局部邻域，也可以是一整行与一整列。

8.2 三个实例的结构对照

表 2 RGB 转灰度、模糊与矩阵乘法的并行结构

计算	输出空间	单线程输入工作集	线程内控制结构
RGB 转灰度	$H \times W$ 灰度图像	同一像素位置的红、绿、蓝三个分量	固定数量的乘加，无循环
邻域模糊	$H \times W$ 模糊图像	以输出像素为中心的有效邻域 $\mathcal{V}_R(r, c)$	两层循环与输入边界条件
矩阵乘法	$M \times N$ 输出矩阵	A 的第 r 行与 B 的第 c 列，共享长度为 K	一层长度为 K 的累加循环

三者的线程数量都由输出空间决定，而单线程工作量由生成一个输出所需的输入数量决定。这一区分有助于判断某个维度应放在网格中，还是应由线程内部循环遍历：

- 输出坐标通常映射到并行线程；
- 单个输出内部的归约维度通常映射到线程内循环。

8.3 关键公式汇总

一维全局下标: $i = \text{blockIdx.x} \text{blockDim.x} + \text{threadIdx.x},$

二维全局坐标: $r = \text{blockIdx.y} \text{blockDim.y} + \text{threadIdx.y},$
 $c = \text{blockIdx.x} \text{blockDim.x} + \text{threadIdx.x},$

二维行主序: $p = rW + c,$

二维块数: $G_x = \left\lceil \frac{W}{B_x} \right\rceil, \quad G_y = \left\lceil \frac{H}{B_y} \right\rceil,$

总启动线程数: $N_{\text{threads,total}} = (G_x G_y G_z)(B_x B_y B_z),$

矩形矩阵乘法: $C_{r,c} = \sum_{k=0}^{K-1} A_{r,k} B_{k,c}.$

8.4 正确性检查表

1. 网格的每个维度是否按对应输出尺寸向上取整？
2. 是否始终使用 x 对应列、 y 对应行的同一约定？
3. 全局坐标是否包含块偏移与块内偏移两部分？
4. 行主序公式中的宽度是否属于正在访问的那个数组？
5. 输出写入之前是否检查了所有输出坐标？
6. 每次输入读取之前是否检查了该输入访问涉及的全部坐标？
7. 邻域循环是否使用能够表示负候选坐标的有符号索引？

8. 边界处的平均值是否除以实际参与求和的元素数？
9. 矩阵乘法中，输出维度 M, N 与归约维度 K 是否各自使用在正确位置？

关键结论

多维网格的核心作用是把数据的几何结构直接编码到线程坐标中。正确核函数由三条关系组成：

线程层次 $\rightarrow$ 逻辑输出坐标, 逻辑坐标 $\rightarrow$ 线性内存地址, 有效性条件 $\rightarrow$ 安全访问.

循环与条件控制单个线程的局部工作集，不改变所有线程共享同一核函数代码这一单程序多数据模型。